code, GraalVM offers a new technology (the Native Image builder) that compiles Java applications directly into machine code, obtaining a *native executable* or *native image* that contains the whole machine code necessary for its execution.

Java applications compiled as native images have faster startup times, optimized memory consumption, and instant peak performance compared to the JVM options. GraalVM builds them by changing the way applications are compiled. Instead of a JIT-compiler optimizing and producing machine code at runtime, the *Native Image* mode is based on *Ahead-Of-Time (AOT) compilation*. Starting from the `main()` method, all classes and methods that are reachable during the application's execution are statically analyzed at build time and compiled into a standalone binary executable, including any dependencies and libraries. Such an executable doesn't run on a JVM but directly on the machine, just like C or C++ applications.

When using native images, much of the work that used to be performed at runtime by the JVM is now done at build time. As a result, building an application into a native executable takes longer and requires more computational resources than the JVM option. The GraalVM AOT compiler does not support some Java features out of the box. For example, reflection, dynamic proxies, serialization, and dynamic class loading require extra configuration to help the AOT compiler understand how to analyze them statically.

How can we adapt existing Java applications to run as native images? How much configuration is required to support frameworks and libraries? How can we provide the necessary configuration for the AOT compiler? That's where Spring Native enters the scene.